

Web Knowledge dump

Jarrett Lane

All about web

-
- Since I am graduating this semester, I want to give you all my web knowledge before I go
 - This will be about both general concepts and offensive concepts and maybe some defensive too
 - This will be a long meeting and you will learn a lot
 - Web takes a lot of studying but it can be very rewarding
 - Throughout this presentation I'll give visual examples

What is a browser

- A browser is software that offers a user friendly way for you to access different web content across the internet
- A browser handles communication, renders web pages, offers built in security function, runs webpage functionality, and more
- When you load a webpage the browser takes what the web server sends to your computer and makes it work/look like a webpage that you see
- We call this the client side

What is a web server

- A server that talks via the HTTP or HTTPS protocol in order to send information to clients or receive data
- When you click a link, you are talking to a specific web server that is sending you all types of data for your browser to handle and format for you
- Web servers are run with backend code that directs how to handle certain requests, this is crucial for basic web functionality
- We call this the server side



What does a web server send

- HTML:
 - This is a basic way of telling the browser how to format a page for a user and what the basic visual structure should look like
- JavaScript:
 - This is code that your browser runs on your computer, this can be for things like handling a user action or maybe running a game
- CSS (Cascading style sheets):
 - Tells your browser how to style the page
- Session token/cookie
 - A piece of data that identifies you to the server everytime you talk to it
- Lots more that I will get into

HTTP/HTTPS protocol

- Basically a standardized way for a client/browser (you) to talk with a webserver/website
- HTTP methods (eg. Get & Post) tell the server what we want to do (send or retrieve data)
- HTTP headers tell the server extra details about the request, this can be the cookie that identifies you
- HTTP data is the actual data itself, this can be the server sending you html or you sending your username/password to log in
- You want this to be encrypted or everyone can see everything you do on the internet clearly, use HTTPS

What is a domain

- Basically a name that identifies a web server at a specific location/ip address, thanks to the DNS protocol
- Instead of having to remember the IP of florida polys web server you just need to look up floridapoly.edu
- When you look up this domain with a browser that means you are talking with a web server that acts as an online representation of an entity online
- A subdomain is a specific part of a website that handles specific content, like a log in page or /login

What is an API

- An API (Application Programming Interface) is a handler that makes two different things work together
- Its like if you send a request to a webstore saying you want to buy a shirt, and the api of the webstore knows that the product id of the shirt is 111, so when you buy the shirt, the api knows to place an order for product id 111
- In this case it handled your request to buy the shirt and made it work with the backend code that processes the purchase

What is a session

- A session is a specified line of communication between 2 things
- HTTP is stateless, this means each request is independent and has no context to past requests
- The server does not know who we are when we talk to it, so we need a way to identify ourselves
- A session is identified with a piece of data called a token/cookie, its like our own identifier
- When you make a request it includes the token/cookie to identify you, and the server will know its you
- The concept of sessions is used in many networking contexts

There's more to a website?



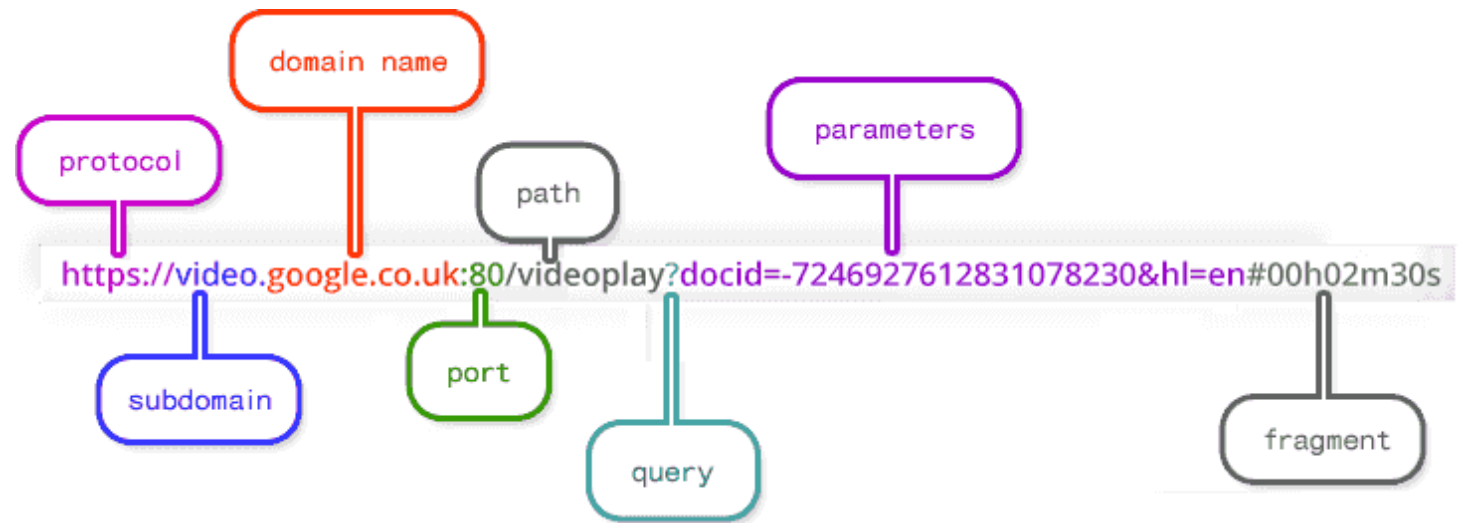
- Many web servers need to work with other services/programs to function
- Common examples include databases, operating system features, file handling, and much more
- Depending on how the backend code handles communication and depending on which packages/tools are used, there are many possibilities for hackers to find a way in or exploit a web server

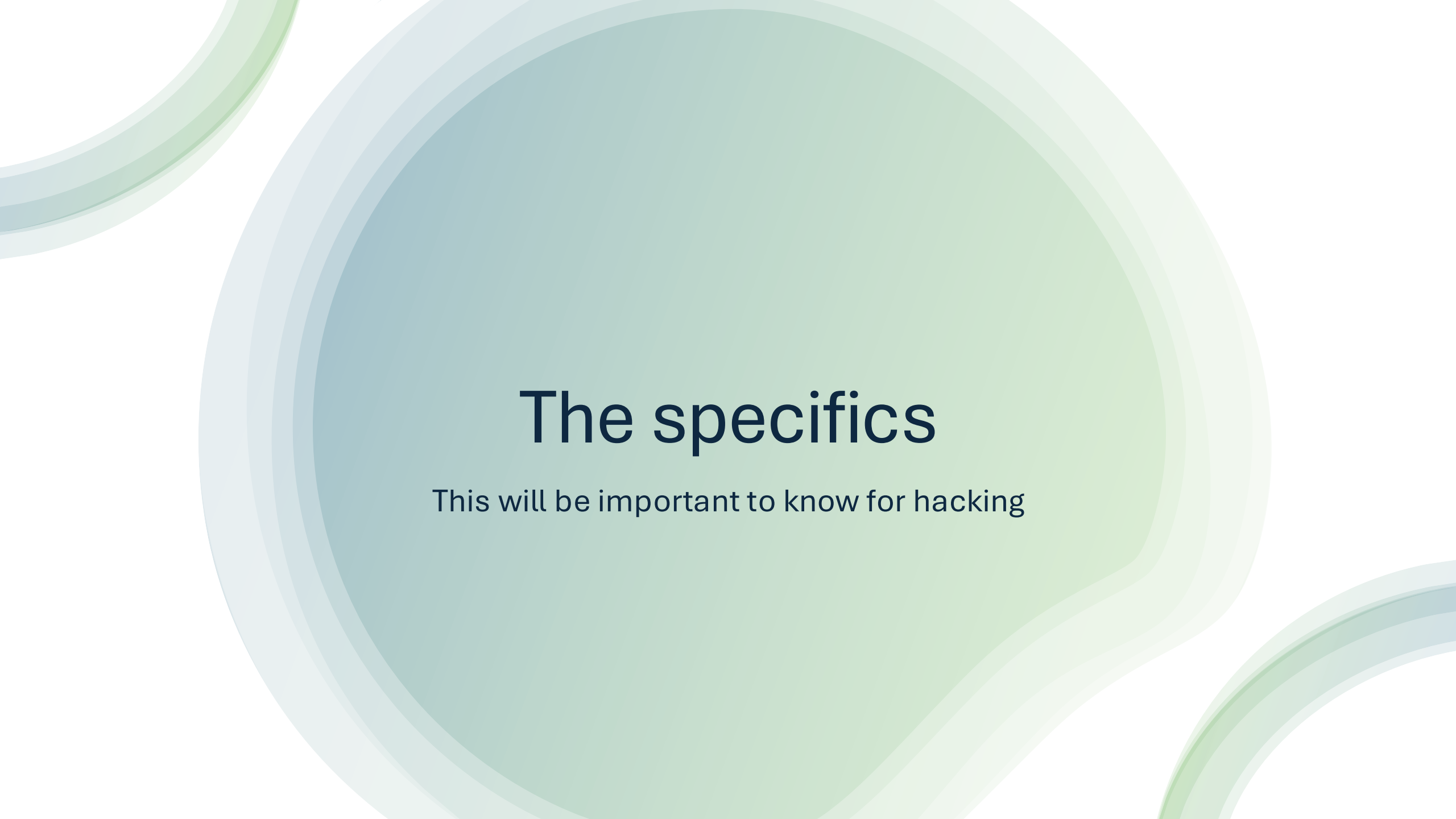
Secure Socket Layer SSL

-
- The S in HTTPS
 - This is a way of using asymmetric key encryption to set up symmetric key encryption
 - Use the public key in the websites certificate to initiate an encrypted handshake, and use that handshake to work out symmetric encryption
 - Kinda hard to explain briefly but that's the basic idea
 - Thanks to this, once your request is sent it's encrypted

What is a URL

- It's the internet address that is comprised of the
 - Protocol (probably https)
 - Domain (who is this?)
 - Subdomain/path (Where on the site?)
 - Vhost (where on the domain)
 - Parameter (data related to the site)
 - Fragment (which part of the specific webpage)





The specifics

This will be important to know for hacking

Backend options

-
- Backend code that runs on the server can be written in many languages such as python, PHP, node.js, and many more
 - There are frameworks/libraries that are used with these languages to develop the backend
 - Each language has its own characteristics, and some of these characteristics can be exploitable if not handled carefully

Document Object Model (DOM)

-
- The DOM is a structure that organizes the web server data in a hierarchy of objects
 - This allows for organization, dynamic interactions and functionality with javascript
 - It basically gives structure to the raw data sent from the web server
 - The DOM is how your browser sees the web content when it parses and renders it

JSON, YAML, XML, etc

-
- Simple formats of data that is easy for humans and computers to read and use
 - Data serialization is just a way of saying something is made simpler, whether it be for storing or transmitting data
 - Many applications (especially web) love these formats because they are a simple way of communicating and structuring data

Databases (SQL, Maria DB, Mongo DB, etc.)

-
- These are database services for storing relevant information to the site (example: which items can be ordered and delivered from this restaurant menu)
 - To talk to/query/update these databases, we use a specific syntax for that program
 - These should be very secure

Inspect element

-
- Basically your browser letting you see the underlying code/data that is making up the website you are seeing
 - You can look at the html structure, debug the running javascript, view communication between you and the site, view your session tokens, and much more

JWT

- Normally the server identifies you from your token by using a system to store which user has which cookie
- With JWT, YOU have a token that identifies you and the server uses that alone to tell who you are
- Whats stopping an attacker from changing the token? Well the server takes the hash of the basic info then encrypts it and adds that to the token
- You have to know the servers key to decrypt it and give the correct hash of the fake data, otherwise the server decrypts the token signature, sees that it does not match the hash of what was sent, and knows its false
- Easier to show than explain: <https://www.jwt.io/>

Web hacking/defending

(mostly hacking)

Web security is versatile

-
- There are many ways that web servers are hacked/exploited, here are some common ways:
 - Unsanitized user input
 - Poorly written backend code
 - Improper access controls/restrictions
 - Outdated tools/packages with known vulnerabilities
 - Inconsistencies between 2 services
 - Session tampering
 - Poor handling of file uploads
 - Weird unexpected behavior
 - Much more
 - I'm going to go over as much as I can

IDOR (Insecure Direct Object Reference)

-
- When you access a certain resource on a website, lets say that's an object
 - Now lets say there are things you shouldn't be able to see, like another users account page
 - What if a website takes lets an attacker access a victims data with user=victim, the website doesn't check anything and shows the attacker all of the victims data

Poorly written code

-
- Sometimes the logic of the code itself is bad
 - These issues can be found by either analyzing the code or by noticing unintended behavior
 - These can also be found by finding creative things that the code doesn't account for
 - This can also just be looking at something on a website and thinking "This is not a safe way of doing this"
 - An example of this is when James hacked the school with permission

Command injection

-
- What an attacker can run commands on a web server via some type of data input, this is command injection
 - This can happen when unsanitized input is passed to a function that is capable of executing commands
 - The best way to prevent this is to try to avoid powerful functions and sanitize user input

SQL injection

- Many websites use some sort of database functionality, and SQL is a common choice
- When a web server talks to a database, it uses a query that either asks for, creates, or deletes data
- If a website takes in data in an unsafe way, an attacker can control the query and control it to do whatever they want
- For example an attacker might manipulate a query to give all of the usernames and passwords in a database
- SQL injection is a huge topic and I would have to teach you all basic sql, its not hard to learn but too much for this meeting

Cross site scripting (XSS)

- Your browser runs javascript to make websites do things
- This javascript comes from the webserver and your browser runs it
- What if an attacker finds a way to trick your browser into running malicious javascript from an attacker?
- In cross site scripting, the attacker is exploiting the fact that they can add malicious javascript to a trusted website in some way
- When a user uses a website, the browser should only run javascript that is necessary for functionality
- A website should protect its users by ensuring that it doesn't let the browser run unexpected javascript

XSS continued

- How does an attacker make malicious code run on a trusted website?
- When your browser renders a web page, it looks for scripts and functions in the web page data and it runs them
- An attacker finds a way to include malicious javascript as part of the over all web content so it gets executed
- An easy example is lets say you load up a blog and in the comments someone puts malicious js code instead of a normal comment
- If the web page does not have xss protection, the browser will assume that comment is part of the javascript of the website and run it

XSS continued

- What could that malicious comment do?
- The attacker could have written javascript that takes the users session token and gives them full access to that users account
- Why did this happen?
- The website didn't set any protections that limit js execution, so a normal comment is treated as code
- How does the website protect its users from this?
- Using a content security policy header tells the browser "You should only be loading/running these resources, do not allow anything else"

XSS continued

-
- Another way to stop malicious javascript from running is to sanitize input that could potentially be interpreted as javascript
 - There are three main kinds of XSS:
 - Reflected
 - Stored
 - DOM

Reflected XSS

-
- Reflected XSS is when malicious javascript is sent in a web request and rendered in the web response
 - Imagine a link like `store.com/?search = *bad js*`
 - The pages shows: "Search results for `*bad js*`" but since the search query is directly rendered in the page it runs like javascript
 - This is often exploited via phishing where a victim clicks a link containing bad code that gets rendered onto the site

Stored XSS

-
- This is like the blog example I mentioned earlier
 - The malicious javascript is stored in the server (maybe via a comment or data field) and when anyone loads that data from the server it executes on their browser
 - Now anyone that loads the data/page containing the bad js gets hacked
 - Common ways of pulling this off are with comments, message boards, profiles, etc.

DOM XSS

-
- This is a type of XSS where the server isn't involved at all
 - DOM xss relies on existing unsafe functions in a web page that are capable of running javascript from some data source
 - This is a bit harder to explain, but imagine an unsafe function that takes data from an attacker controlled source, if a user is tricked into clicking a link using the attackers payload, the payload is sent to the unsafe function and executes on the clients browser

Protect yourself from XSS

-
- Don't click random links
 - Using updated modern browsers for the latest security features
 - Use reputable security extensions
 - This is a bit radical but you can use an extension called noScript that basically blocks all javascript from running unless you specify otherwise, this can be a bit much for day to day use but its up to your preference

Outdated tools/packages

- Webservers often use many other functionalities to operate, sometimes, an outdated package or tool being used on the website will have a known vulnerability/bug that can be exploited
- Attackers can find these cases by using scanning tools to identify the services running on a server, or by leaking information via error codes that point to a certain outdated version
- In CTF it is important to pay attention to the versions of libraries and see if there are any exploitable vulnerabilities
- The best way to prevent attacks is to keep tools/packages updated, especially if a vulnerability has been found with the current used version

Inconsistencies between different things

-
- Sometimes, data can be interpreted in many different ways by different systems
 - There are many ways data is encoded and interpreted
 - Many of these vulnerabilities are seen with the integration of different programming languages that have their own data types and interpretations
 - Sometimes security checks don't cover certain input that another program actually uses
 - An example of this is http parameter pollution

Session tampering

- Session tampering is finding a way to fake your session with the server, allowing attackers to pretend to be another user
- Session tampering usually involves finding ways to steal a token that is assigned to another user (XSS is a great way to do this)
- With JWT, session tampering comes down to either leaking the key, lack of token checks, or other JWT specific vulnerabilities
- Stealing sessions bypasses password and MFA because the server will always associate that session with a valid user that already logged in

File uploads

- File uploads are common in websites, this can be for things like profile pictures or even submitting homework
- Depending on how the server handles these files, it could be possible to upload and run malicious files
- A common attack that abuses file uploads is reverse/bind shell, we did a meeting on this
- Also files like pdf and xml files (and more) have their own vulnerabilities and can be used in attacks

Conclusion

References and stuff



These slides aren't enough

- I could go on and on and elaborate way more on things
 - I probably forgot a few things too
 - I hope what I said is at least enough to give you all a basic understanding
 - Web security covers so many concepts and it can get complex pretty fast, but don't let that scare you
 - The more you participate in CTF, read writeups, study, etc. You will start to learn a lot
-

Resources

- I'll be honest its hard to gather every single resource I've used
- https://www.youtube.com/watch?v=4Jk_I-cw4WE&list=PLI_rLWXMqpSl_TqX9bbisW-d7tDqcVvOJ
- <https://portswigger.net/web-security/all-topics>
- W3 schools
- <https://developer.mozilla.org/en-US/>
- GeeksForGeeks
- Much more